Experimento A · Ío en Minecraft

Resumen de lo conseguido, hallazgos y oportunidades de mejora · 13 de octubre de 2026

1. Objetivo

Construir y entrenar un **modelo propio, pequeño**, que se encargue de la movilidad y las decisiones de un cuerpo (un bot) dentro de un mundo sandbox, con dos modos —**órdenes** y **libre**— y con «propósitos» intercambiables. La idea de fondo: estudiar cómo se comporta un agente en un mundo simulado y, más adelante, qué efecto tiene el propósito que se le da.

50 %
aciertos en Minecraft (15/30, banco repetible)

20→50 %
evolución en las mediciones de 20-30 pruebas

76-88 %
aciertos en el simulador, según versión

0 claves
sin API de terceros: el modelo decide en local

2. Lo que se ha construido

Pieza	Qué hace
Servidor local	Minecraft Java 1.21.11 oficial (verificado con su firma), solo en 127.0.0.1, modo offline, dificultad pacífica. RCON local activado para teletransportar al bot en las pruebas (clave aleatoria en un archivo privado).
Cliente	TLauncher extraído dentro del proyecto, sin instalarlo en el sistema; trae su propio Java 21.
Cuerpo (bot)	Bot Mineflayer 4.39 con puerto de control local. Órdenes: estado, percibir, decir, ven, para, ir, talar, árboles, cerebro. Se reconecta solo y limita el tiempo de cada movimiento.
Mente de Ío	Memoria y diario en archivos de texto, con 11 ciclos documentados (qué percibió, qué hizo, qué aprendió).
Simulador	Mundo de cuadrícula (8 a 24 casillas) con obstáculos en manchas, que genera siempre problemas resolubles. Es la «gimnasia» del modelo.
Cerebro	Red neuronal de 2 capas ocultas de 64 neuronas entrenada por refuerzo (DQN) con PyTorch en CPU: entrena en unos minutos en el propio equipo. También hay cuaderno para Google Colab.
Adaptador	Traduce lo que ve el bot en Minecraft a las mismas entradas que el simulador y ejecuta la acción elegida: avanzar, girar izquierda, girar derecha o picar.
Banco de pruebas	30 pruebas repetibles: teletransporte por RCON a puntos fijos (semilla 42) a 12-30 bloques del arbolado; éxito = el bot pica un tronco.

3. Cómo evolucionó el rendimiento

Versión del modelo	Simulador	Minecraft
Árbol, sin obstáculos, 3 entradas	100 %	Se atascaba en obstáculos reales
+ obstáculos y entrada «bloqueado»	67 %	Camina hasta ~20 casillas
+ memoria de la acción anterior	86-88 %	2/3 (muestra pequeña)
Mismo, 20 pruebas variadas	—	**4/20 (20 %)**
Rejillas 8-24 con terreno en manchas	76 %	5/13 válidas (38 %)
+ penalización por giros repetidos	65-76 %	3 de ~10 válidas; banco poco fiable
Banco repetible, bot sin cuelgues	—	**13/30 (43 %)**
+ corrección: picar al aire no termina	—	**15/30 (50 %)**

La política escrita a mano (referencia) acierta el 51-57 % en el simulador con obstáculos: el modelo la supera, pero no por un margen enorme.

4. Hallazgos principales

- **La brecha simulador → mundo real es el problema central.** Cada fallo en Minecraft fue algo que el simulador no tenía: un árbol en la propia columna, troncos altos, distancias mayores de lo entrenado, desniveles, hojas. Cada uno se corrigió trasladándolo al simulador o al adaptador.
- **Medir mal engaña.** Con 3 pruebas salió un 67 %; con 20 salió un 20 %. Varias mediciones estuvieron contaminadas por cuelgues del navegador del bot y desconexiones. Solo el banco repetible con teletransporte (30 pruebas, semilla fija) permite comparar versiones.
- **Sin memoria, el agente oscila.** Con solo lo que ve en el instante, gira a un lado y a otro sin avanzar. La memoria de la acción anterior elevó el simulador del 67 % al 86 %.
- **Los obstáculos reales son ahora el cuello de botella:** 13 de los 15 fallos de la última medición incluyen choques repetidos contra algo que bloquea (hojas, troncos, desniveles de 2 o más).
- **Coste y autonomía:** el modelo propio decide sin consumir tokens ni claves de terceros; la intervención de Ío (planificar, depurar) se limita a lo imprescindible.

5. Oportunidades de mejora

Corto plazo (el siguiente paso inmediato)

- **Visión lateral:** dar al modelo la casilla libre/bloqueada a izquierda, frente y derecha para que sepa por dónde rodear. El simulador y el adaptador ya están preparados; **el entrenamiento se interrumpió** y hay que relanzarlo y volver a medir con las mismas 30 pruebas.
- **Mejor detección de obstáculos:** distinguir hojas (que el navegador rodea) de paredes, y desniveles reales.
- **Recoger el tronco** tras picarlo (ahora solo lo pica).
- Más memoria: historial de 3-4 acciones, no solo la última.

- Subir a 50-100 pruebas con intervalos de confianza antes de afirmar una mejora.

Medio plazo (fases 2 a 4 del plan)

- **Habilidades de alto nivel** (estilo Voyager): ir al árbol, picar, recoger, ir al jugador, huir. El modelo elige habilidades y no pasos sueltos, lo que acelera mucho el aprendizaje.
- **Subobjetivos con el navegador:** que Mineflayer acerque al bot a unas siete casillas y la red resuelva solo el tramo final.
- **Perfiles de propósito** (cooperativo, supervivencia, explorador, constructor): archivos que entran al modelo como entrada extra y modulan la recompensa.
- **Interruptor órdenes / libre**, con registro de cada decisión en el diario.

Largo plazo (investigación)

- Experimentos comparativos: mismo mundo y semilla, un perfil por ejecución; métricas de recursos, supervivencia, distancia y cooperación con el jugador.
- Variantes: ruido en la percepción, metas que cambian a mitad de camino, varios agentes.
- Conectar con la línea inicial de la conversación: autómatas (juego de la vida) y teoría de conjuntos en un sandbox, con el simulador propio como banco previo.

6. Riesgos y límites

- Mineflayer soporta hasta la 1.21.11 (la estable más reciente es la 26.3): no actualizar el servidor sin comprobarlo.
- El servidor debe seguir siendo local y offline; el modo offline no debe exponerse a internet.
- El modelo es de decisión instantánea (más memoria corta): no planifica a largo plazo. Para eso sirven las habilidades y los subobjetivos.
- TLauncher es un launcher de terceros no oficial: se descargó de su web y no se instaló en el sistema.

> **Estado actual a tener en cuenta:** el adaptador del bot ya calcula 10 entradas (con visión lateral), pero no hay una política entrenada compatible porque el entrenamiento se detuvo. Hasta relanzarlo, el bot usa la política de referencia escrita a mano. Las versiones anteriores del modelo están guardadas en `entrenamiento/politica_v*.json` (de 3 a 8 entradas; no son compatibles con el adaptador actual).

7. Dónde está todo

`laboratorio/mundo-simulado/PLAN.md`	Plan por fases y estado
`mente/diario.md`, `mente/memoria.md`	Diario de ciclos y memoria de Ío
`bot/io.js`	Cuerpo, adaptador del cerebro y puerto de control
`entrenamiento/entorno.py`, `entrenar.py`, `entrenar_io.ipynb`	Simulador, entrenamiento local y cuaderno de Colab
`servidor/`	Servidor, `arrancar.sh` y cliente RCON

Generado por Ío a partir del diario y el plan del proyecto. Las cifras de Minecraft proceden de las mediciones registradas; las de 3 pruebas son anecdóticas y se incluyen solo como historial.